Great! Here's the full presentation script for the next speaker:

Speaker 10: Susanta (again)

Topic: Security – How we protect user data and app access

Script (Presentation Story)

Thank you, Aakash.

Now I'll explain a very important part of our project — **Security**.

Since our app handles **user login, personal chat, and room data**, we took extra care to make sure that everything is **safe, private, and secure**.

Let me explain the different layers of security we implemented in **MessFinder**.

Secure Login (Firebase Authentication)

We used **Firebase Auth** for managing user accounts.

- It supports **email/password and Google login**
- Passwords are never stored by us — they're encrypted and managed securely by Firebase
- We don't allow unauthorized users to access protected areas like chat or submit pages

This prevents bots or strangers from misusing the system.

Secure Real-time Chat

The messaging system is built using **Firestore**, which:

- Stores each chat in a private subcollection
- Ensures that only the **two people involved** in the chat can read or write those messages
- Syncs data in real time using SSL (secure socket layer)

This makes the communication safe and fast.

Role-Based Access Control

We created **two separate roles**: user and owner.

- This role is stored in Firestore when a person signs up
- We use this role to **restrict access to certain pages**
    - Example: only owners can see the Submit PG page
    - Only users can see the Room Search page

This ensures that users can **only perform actions they're allowed to**.

Data Access Control via Firestore Rules

Firestore provides a set of **custom security rules**.

For example:

```
match /rooms/{roomId} {
  allow read: if true;
  allow write: if request.auth.uid == resource.data.ownerId;
}
```

This rule ensures that:

- Anyone can view rooms
- But only the **owner who posted it can edit it**

We also secured messages and user data using similar logic.

HTTPS and Secure Hosting

- Our app is hosted on **Vercel with HTTPS enabled by default**
- HTTPS encrypts all communication between the **user's browser and our servers**
- This protects against **man-in-the-middle attacks** and data leaks

---

Summary:

In total, we implemented five security layers:

- Firebase Auth
- Firestore-secured chat
- Role-based access

- Database-level security rules
- Encrypted hosting via HTTPS

This keeps the system clean, private, and trustworthy for both seekers and owners.

Now, I'd like to invite **Sujoy** again to present the final part — **our future plans and what's next for MessFinder**.

Tips for Susanta:

- Speak with authority — this is a serious and technical topic
- Use phrases like "we made sure", "we enforced", "we restricted"
- Keep code snippets brief — only for demonstration
- Pause between each layer of security

Likely Viva/Panel Questions for Susanta:

Q: Can users access each other's chat messages?
A: No. Each chat document is uniquely stored and rules prevent unauthorized reads.

Q: Can someone bypass roles by editing the frontend?
A: Frontend roles are backed by Firestore and Firebase rules — so even if someone changes UI locally, they can't write unauthorized data.

Q: What happens if someone tries to directly access a Firestore collection?
A: Without a valid token and permission based on their role, the request will be denied by Firebase security rules.
